Course 1 of 3

# Fundamentals of Software Architecture for Big Data

Software Architecture for Big Data Specialization
University of Colorado Boulder

Compiled by Ioana-Raluca Tiriac

# Contents

# 1 Module 1 — Software Engineering & Design

## 1.1 No Silver Bullet

*"The Essence and Accidents of Software Complexity"* — Frederick P. Brooks Jr.

Brooks' famous thesis is that there is no "silver bullet" — no single technological or management development that will provide an order-of-magnitude (10×) improvement in software productivity, reliability, or simplicity within a decade. This is because the *accidental* difficulties of software have largely been solved, leaving only the *essential* difficulties inherent in the nature of the software itself.

> **Essence vs. Accident**
>
> - **Essence: the inherent difficulties of fashioning complex, abstract conceptual structures (data sets, algorithms, functions).**
> - **Accidents: difficulties that attend the production of software but are not inherent to it — awkward programming languages, hardware constraints, slow turnaround.**
>
> **Past breakthroughs (high-level languages, time-sharing, unified environments) boosted productivity by removing *accidental* barriers. Because the conceptual essence now takes up most of development time, further tooling improvements only yield marginal gains.**

### 1.1.1 The four essential difficulties of software

1. **Complexity.** Software entities are more complex for their size than perhaps any other human construct. No two parts are alike; if they were, they would be merged into a single subroutine. Scaling software isn't just repetition of parts — it increases the number of distinct elements and their non-linear interactions.
2. **Conformity.** Unlike physics, where one expects unifying principles, software must conform to arbitrary human-designed interfaces, institutions, and legacy systems. This complexity is forced by external requirements and cannot be simplified away by redesign.
3. **Changeability.** Software is under constant pressure to change because it embodies the system's function and is perceived as "pure thought-stuff" — infinitely malleable. Successful software is pressured to change further as users find new applications or as it is ported to new hardware.
4. **Invisibility.** Software has no ready geometric representation. Diagramming it produces several overlapping, non-planar graphs (data flow, control flow, etc.), making it hard to grasp the big picture or communicate it to others.

### 1.1.2 Promising attacks on the essence

- **Buy vs. Build.** The most radical solution is not to construct software at all — the mass market makes it much cheaper to buy and adapt than to build custom from scratch.
- **Requirements refinement & rapid prototyping.** The hardest part is deciding precisely what to build. Because clients often don't know what they want until they see it, use iterative prototyping to make conceptual structures "real" and refine requirements early.
- **Incremental development ("grow, don't build").** Grow software organically: start with a running system that does very little (calling "dummy" subprograms) and flesh it out bit by bit, keeping a working system at every stage.
- **Great designers.** The gap between a good and a great design is often an order of magnitude. Organizations must identify and nurture great conceptual designers through mentoring, career development, and interaction with other top designers.

### 1.2 Simple Aged Cache — TDD & the Tech Stack

These materials introduce test-driven development and the technical framework used for the assignments. Test early and often, using the **red–green–refactor** cycle to keep code high-quality and maintainable.

- **Testing levels:** unit, integration, and acceptance tests.
- **Test doubles:** mocks and fakes help isolate logic under test.
- **Loose coupling:** achieved through **dependency injection** and the **inversion of control** principle.
- **Tech stack:** a practical JVM stack using Java, Kotlin, and Gradle.

> **Acceptance tests**
>
> **Defined from the perspective of the person using the software. They represent the specific requirements or criteria a product owner would need to "accept" to consider the software complete.**

## 2 Module 2 — Fundamentals of Software Architecture

### 2.1 Blockchain

> **Blockchain**
>
> **Ordered, distributed data storage verified by cryptography. Though commonly associated with Bitcoin, it is a general technology for securing data without a central authority.**

#### 2.1.1 Core structure and security

- **Blocks and hashes:** each block contains data, a timestamp, and a cryptographic hash.
- **The chain:** each block includes the hash of the previous block. If any earlier data changes, its hash changes, breaking the entire chain and making tampering evident to all.
- **Hash properties:** a hash function is a "true function" (same input always yields the same output) where even a tiny change in data produces a completely different hash.

#### 2.1.2 The mining process and proof-of-work

- **Mining:** the process of adding new blocks. Miners compete to find a hash with specific properties — one that starts with a pre-defined number of zeros.
- **The nonce:** since the block's data and previous hash are fixed, miners add and increment a **nonce** to change the resulting hash until it meets the "zero bits" requirement.
- **Difficulty adjustment:** in Bitcoin, difficulty is adjusted roughly every 2,000 blocks so the average mining time stays around 10 minutes, providing stability.

#### 2.1.3 Bitcoin as a peer-to-peer electronic cash system

- **Solving double-spending:** allows online payments to be sent directly between parties without a financial institution, using a peer-to-peer network to create a public transaction history.
- **Digital signatures:** transactions form a chain of digital signatures — each owner signs a hash of the previous transaction and the public key of the next owner.
- **Network consensus:** nodes always treat the longest chain as correct, since it represents the greatest proof-of-work effort.
- **Incentives:** the creator of a block is rewarded with new coins and transaction fees, encouraging nodes to support the network and stay honest.

#### 2.1.4 Efficiency and privacy

- **Reclaiming space:** transactions are hashed in a **Merkle Tree**, so old transactions can be pruned while keeping the block's root hash intact — users need only keep block headers for simplified payment verification.
- **Privacy model:** unlike banks (which restrict access to information), Bitcoin keeps public keys anonymous. The public can see transactions but cannot easily link them to individuals.

> **Security calculation**
>
> **Security assumes honest nodes control a majority of CPU power. The probability of a dishonest attacker catching up diminishes exponentially as more blocks are added. If an attacker controls 10% of network power, the probability of catching up after 5 blocks is only about 0.1%.**

**Merkle Trees.** Attributed to R. C. Merkle (1980, "Protocols for public key cryptosystems"), they serve three roles here:

- **Reclaiming disk space:** only the root hash need be included in the block hash, so old spent transactions can be pruned without breaking cryptographic integrity.
- **Efficient verification:** "stubbing off" branches enables **Simplified Payment Verification (SPV)** — verify a transaction using only the Merkle branch linking it to the root, rather than downloading the whole block.
- **Visual representation:** transactions ($Tx_0, Tx_1, \dots$) are hashed into a tree forming a single root hash in the block header.

### 2.2 Software Architecture — The Application Continuum

A philosophy of **evolutionary architecture**: evolve software from a simple monolith into a distributed system based on the amount of information available. The goal is software that is easy to read, adapt, and extend, so teams can embrace change.

> **Three fundamental programming tasks**
>
> - **Naming things accurately as they are.**
> - **Organizing codebases so naturally related items are grouped together.**
> - **Reducing circular dependencies, which become a major hurdle as systems grow.**

#### 2.2.1 The continuum: steps of evolution

Often represented as a series of Git commits:

1. **Flat directory (v1):** no big upfront design — one directory to avoid "analysis paralysis" and get a working example quickly.
2. **Functional groups (v2):** organize by technical role (Controllers, Models). Common in MVC, but leads to "low feature cohesion" — small changes ripple across directories.
3. **Feature groups / bounded contexts (v3):** group by domain (e.g. "Allocations", "Backlog"). Improves readability; a step toward loosely coupled components.
4. **Component-based architecture (v4):** extract components built and tested independently. Build tools (Gradle) declare dependencies and prohibit circular ones.
5. **Multiple applications (v5):** split into distinct applications, often kept in a single Git repository (monorepo) to reduce overhead and simplify cross-component refactoring.

6. **Microservices & distributed systems (v6–v10):** service-to-service REST, a database-per-service model, and distributed patterns like service discovery and circuit breakers.

**The "Monolith First" strategy**

- **Success rates: most successful microservice stories began as monoliths later broken up; systems built as microservices from scratch often face serious trouble.**
- **The "microservice premium": microservices carry a management cost that can slow a team. For new products with low product–market fit, speed and cycle time beat early scaling.**
- **Stable boundaries: correct bounded contexts are hard to identify early — a monolith lets boundaries stabilize before they are frozen into separate services.**

**Strategic considerations.**

- **Starting point** depends on information: startups with high uncertainty start far left (monolith); enterprise rewrites of known systems may start further right.
- **Technical debt:** following the continuum helps find the right "seams" and boundaries, avoiding a total rewrite to escape accumulated debt.
- **Legacy:** the goal is to leave a "good legacy" — code so well-organized that future developers find it "awesome" rather than a burden.

References: `https://www.appcontinuum.io/` repo: `https://github.com/initialcapacity/application-continuum`

### 2.2.2 The 10-step journey in four phases

**Phase 1 — Single app (v1–v2)**
Start flat, then organize by technical function (models, controllers, DAL). Functional grouping is an anti-pattern — changes to one feature ripple everywhere — so v2 is reverted in favor of v3.

**Phase 2 — Namespaced monolith (v3–v4)**
Reorganize by bounded context (`allocations/`, `backlog/`, `timesheets/`) instead of technical type. v4 extracts these into independently buildable components with explicit Gradle dependencies — this is where circular dependencies get killed. Key call: `JdbcTemplate` over JPA/ORM to avoid the bidirectional dependency trap.

**Phase 3 — Multiple apps (v5–v7)**
v5 deploys 4 applications (Allocations, Backlog, Registration, Timesheets), still in one repo but independently deployable. v6 introduces service-to-service REST. `Info` and `Client` classes are intentionally duplicated across services rather than shared — a deliberate trade-off enabling independent versioning. v7 gives each service its own database, completing full decoupling.

**Phase 4 — Distributed systems (v8–v10)**
v8 adds API versioning via `Accept` headers (Registration's Projects gets a v2 for "funded projects" — Timesheet uses it, others stay on v1). v9 replaces hardcoded service URLs with Redis-backed client-side service discovery (30s heartbeats). v10 adds the **circuit breaker** pattern — if a downstream is slow or unreachable, fail fast via a fallback rather than cascading the timeout.

**Design decisions woven throughout.** Avoid `common/util` packages (name things for what they do); never commit config/credentials; keep test packages different from source packages

to enforce encapsulation; and separate `Record/Info` classes to keep the public API decoupled from the DB schema.

#### 2.2.3 Aside: the Saga pattern

- **Choreography-based saga** (event-driven): typically uses an event store or message broker (Kafka, RabbitMQ). A relational DB alone isn't enough — you need somewhere to publish and consume events reliably.
- **Orchestration-based saga** (no event store): a central orchestrator coordinates the flow via direct calls (REST or otherwise). Each service keeps its own relational DB; the orchestrator tracks state and issues compensating transactions on failure.

#### 2.2.4 The circuit breaker

> **The problem: cascading failure**
>
> **When the Registration service goes down and Timesheets keeps calling it, every request hangs, threads pile up, and Timesheets itself slows and crashes — one broken service becomes two. The circuit breaker is modeled on an electrical breaker: too many failures trips it, cutting the connection to protect the rest of the system.**

**Closed**
Normal operation. Every request goes through; failures are counted. Once failures hit the threshold (default 3), the circuit trips open.

**Open**
The circuit has tripped. No requests reach the downstream — the fallback is called immediately, so threads don't pile up on timeouts. After a wait (default 5s) it moves to half-open.

**Half-open**
One probe request is allowed. Success → circuit closes and normal operation resumes. Failure → circuit opens again and the wait restarts. This prevents flapping.

The **fallback function** is whatever the app does instead of the real call — a cached result, an empty list, a degraded UI, a logged error. In the repo's v10 this is implemented with a `ScheduledExecutorService`: the real call runs as a task with a timeout, and a task that doesn't finish in time is treated as a failure.

# 3 Module 3 — Fundamentals of Production Software

## 3.1 The production-readiness mindset

- **Production first:** focus on production early rather than as an afterthought — a common strategy is to deploy a "Hello World" with minimal functionality and iterate once it's live.
- **Cloud-native MVP:** should include basic monitoring, a hint of scalability, an initial SLA, and blue–green deployments for zero downtime.
- **Monitor early:** like testing, monitoring and metrics should be baked in early to surface bottlenecks (long DB transactions, slow API calls).

## 3.2 Service level definitions (SLIs, SLOs, SLAs)

**SLI — Service Level Indicator**
Quantitative measures of service level: meters (speed of requests), gauges, counters (how many events occurred), health checks.

**SLO — Service Level Objective**
Precise numerical targets for availability and responsiveness (e.g. up 99.99% of the time).

**SLA — Service Level Agreement**
A contract with users defining what happens if SLOs are not met.

> **The myth of 100%**
> **Aim for "almost-perfect" reliability (99.99% or 99.999%) rather than 100%: users can't distinguish the two because of failures in their own environment (home Wi-Fi, ISPs).**

## 3.3 Availability mathematics

$$\text{Availability} = \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}$$

where MTTF is Mean Time To Failure and MTTR is Mean Time To Repair.

- **Rule of the extra 9:** a service cannot be more available than its critical dependencies. To hit a target, each unique critical dependency should offer one additional "9" ($10\times$ more reliable) than the service itself.
- **Levers for reliability:** (1) reduce *frequency* of outages (testing, rollout policies); (2) reduce *scope* (sharding, isolation); (3) reduce *time to recover* (monitoring, automated rollbacks).

## 3.4 Error budgets

> **Error budget**
> **An error budget is $1 -$ SLO (a 99.99% SLO leaves a 0.01% budget for downtime). As long as the budget isn't spent, teams can launch new features; if it's exhausted, launches freeze until it resets. This gives SRE and development a shared, data-driven metric and removes tension between them.**

## 3.5 Design strategies for reliability

- **Eliminate SPOFs:** identify and remove single points of failure.
- **Redundancy & isolation:** multiple independent instances or geographic isolation so failures don't spread.
- **Asynchronicity:** decouple from noncritical dependencies with async calls so a dependency's latency doesn't hurt the parent.
- **Graceful degradation:** be "elastic" — offer limited functionality rather than a total error page.
- **Failover & fallback:** automate them, since human intervention is often too slow for tight SLOs.

## 3.6 Tools and testing

- **Tooling:** Prometheus for collection, Grafana for visualization, DataDog for log and metric surfacing.
- **Fault injection:** use integration tests to deliberately inject faults and verify the system survives dependency failures.
- **Overload testing:** deliberately overload to see how the system degrades before users do.
- **Progressive rollouts:** deploy to small percentages of users (5/20/100%) to minimize impact if a bug slips in.

# 4 Module 4 — Databases & Messaging Systems

## 4.1 Working with relational databases

**CAP theorem**

**If a system is subject to communication failures, it is impossible to provide consistency and availability simultaneously — you must trade off.**

- **Consistency (C): every server returns the correct response, preserving request order.**
- **Availability (A): every request eventually receives a response, with no strict time limit.**
- **Partition tolerance (P): the system keeps working despite communication failures between nodes.**

A common proof uses two nodes $G_1$ and $G_2$ that cannot talk (a partition). If a write happens on $G_1$, then $G_2$ still holds the old value. To be *available*, $G_2$ must answer a read — returning inconsistent data. To be *consistent*, $G_2$ must wait for the partition to heal — making the system unavailable.

**The Milk Problem**

**A grocery chain used Cassandra (eventually consistent, favoring availability). Because multiple stores sent updates to a central server simultaneously, the system returned inaccurate counts (e.g. 130 units when the true count was 126) because it wasn't atomic. Fix: for inventory accuracy, consistency matters more than speed — switching to a relational database with transactions (select, update, commit in sequence) ensures accurate counts at the cost of some availability under extreme load.**

### 4.1.1 Evolutionary database design practices

- **Collaborative culture:** DBAs and developers pair closely to understand both the global data view and specific application needs.
- **Version control:** all database artifacts (schema, scripts, standing data) live in the same repository as the application code, so the database is never out of sync.
- **Automated migrations:** every change is a migration script (DDL for schema, DML for data). Tools like Flyway or Liquibase use a "changelog" table to track which numbered scripts have been applied.
- **Individual database instances:** every developer and QA member gets a private instance to experiment without disrupting others.
- **Continuous integration:** integrate DB changes into the mainline at least daily; a server builds the database from scripts and runs the full suite to catch conflicts early.
- **Database refactoring:** small internal-structure changes (e.g. splitting a table) that don't change observable behavior.
  - *Destructive changes* (e.g. making a column non-nullable) need extra care.
  - *Transition phase:* for shared DBs, support both old and new access patterns at once (e.g. a View simulating a renamed table) while clients migrate.

**Key concepts**

- **Mainline: the shared version-control repository where all integrated changes live.**
- **Standing data: necessary data (e.g. a list of countries) that must be version-controlled**

**alongside the schema to enable testing.**

- **Frequency reduces difficulty: many small, frequent integrations are easier than one large, infrequent one.**
- **Schemaless vs. implicit schema: even "schemaless" NoSQL databases have an implicit schema defined by the code that accesses them, which still needs migration management.**

## 4.2 Working with message systems

Introduced by Eric Brewer in 2000, CAP identifies a fundamental trade-off between consistency, availability, and partition tolerance in a distributed web service. In a network subject to communication failures, it is impossible to guarantee both consistency and availability: a server in a partitioned network must either risk an incorrect response (sacrificing consistency) or refuse to respond (sacrificing availability).

**Safety vs. liveness**

**CAP is a specific instance of a broader trade-off:**

- **Safety — "nothing bad ever happens." Consistency is a safety property (every response is correct).**
- **Liveness — "eventually something good happens." Availability is a liveness property (every request eventually gets a response).**

**The related FLP impossibility result (Fischer, Lynch, Paterson) proved that fault-tolerant consensus is impossible in an asynchronous system if even one process fails — showing safety and liveness are impossible to guarantee even with simple crash failures and no partitions.**

### 4.2.1 Practical strategies and case studies

- **Prioritizing consistency (CP):** "best-effort availability" but prioritize correctness. *Example — Chubby Lock Service:* used by Google for distributed coordination via the Paxos protocol; under a partition it becomes unavailable rather than return incorrect data.
- **Prioritizing availability (AP):** guarantee a response but provide "best-effort consistency." *Example — web caching (Akamai):* content served from nearby servers for speed; updates take time to propagate, so users may see different versions.
- **Event collaboration & messaging:** combining a database with a messaging system (e.g. RabbitMQ) often uses a two-phase commit. A failure between the DB commit and the messaging transaction creates an edge case where it is unknown whether the DB record actually changed. *Compromise:* for low-stakes tasks (counting milk) use one-phase commits and retries, accepting small error; for critical tasks (911 calls, contact tracing) consistency cannot be sacrificed.

### 4.2.2 Techniques for coping with trade-offs

- **Segmentation:** split the system rather than applying one uniform trade-off. *Data partitioning* — a shopping cart is highly available (AP) while billing is strongly consistent (CP). *Operation partitioning* — reads highly available, writes strongly consistent.
- **Continuous consistency:** toolkits like TACT let designers specify exactly how much inconsistency is acceptable (a unit called *conits*) — e.g. an airline system allows more inconsistency when many seats are free, tightening as the plane fills.
- **Shared databases:** a messaging system can use the same backing database as the storage

layer, so both commit in a single transaction. This guarantees consistency but is slower and reduces availability.

Course 2 of 3

# Software Architecture Patterns for Big Data

Software Architecture for Big Data Specialization
University of Colorado Boulder

Compiled by Ioana-Raluca Tiriac

# Contents

# 1 Module 1 — Predictive Models

## 1.1 Prediction models

Predictive modeling is a statistical process that analyzes patterns in data to predict future events. Though often used interchangeably with machine learning, predictive modeling specifically refers to the statistical technique that *uses* ML models. Two broad categories:

- **Supervised learning:** requires a labeled input data set.
- **Unsupervised learning:** processes unlabeled data, relying on traditional statistics or neural networks to distinguish patterns.

**Best practice: models are code**

**Integrate models into the system's source code:**

1. **Create an interface so the code can store and use different models per situation.**
2. **Use source control to track changes over time and prevent data loss.**
3. **Write unit tests — fast, isolated, and repeatable — so each model behaves correctly.**
4. **Generate reports — measurement reports for models across datasets.**
5. **Integration tests to ensure the model works within the whole system.**

## 1.2 Evaluation metrics and formulas

Metrics depend on whether the task is **classification** (categorizing results) or **regression** (predicting actual values).

### 1.2.1 General accuracy

Simple, but not always a holistic view of performance:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Predictions}}$$

### 1.2.2 Classification metrics

These capture how well a model identifies truly relevant outcomes.

$$\text{Precision} = \frac{TP}{TP + FP} \qquad \text{(how many predicted-positive are truly relevant)}$$

$$\text{Recall} = \frac{TP}{TP + FN} \qquad \text{(how well it finds all truly relevant outcomes)}$$

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \qquad \text{(harmonic mean of precision and recall)}$$

where $TP$ = true positives, $FP$ = false positives, $FN$ = false negatives.

### 1.2.3 Regression metrics

Measuring the average difference between predicted values $x_i$ and true values $y_i$:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{n}(x_i - y_i)^2 \qquad \text{(penalizes large errors more heavily)}$$

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - y_i)^2} \qquad \text{(used when MSE gets too large to compare)}$$

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{n}|x_i - y_i| \qquad \text{(weights all errors proportionally)}$$

**Confusion matrix**

**A tool to visualize accuracy per class, allowing a comparison of performance across different classes in a binary classification problem.**

## 1.3 Building and evaluating a prediction model

The **match predictor** is a full-stack application that predicts soccer match outcomes.

- **Front end:** TypeScript and React.
- **Back end:** Python and Flask.
- **Data source:** historical results from 538 (fixtures, outcomes, goal counts).
- **Core interface:** all predictors follow one interface — a `predict` function takes a fixture (two teams, their names, and leagues) and returns a prediction.
- **Prediction components:** an *outcome* (home win, away win, or draw) and an optional *confidence* percentage.

### 1.3.1 Built-in predictors (benchmarks)

- **Home predictor:** always predicts the home team wins.
- **Past results / points predictor:** predicts based on which team accumulated the most points or wins in past games.
- **Linear regression predictor:** uses Scikit-Learn to build a predictive model.
- **Simulation predictors:** run background simulations of how likely each team is to score during any given minute of a match.

### 1.3.2 Evaluation and testing framework

- **Unit tests** (`make backend/test`) — fast-running checks of framework correctness.
- **Accuracy measures** (`make measure`) — slower tests that train models and verify they meet thresholds (e.g. above 33% or 37%).
- **Reports** (`make report`) — train on past seasons (2019–2020) and test on a more recent season (2021) across different leagues.

**Overfitting vs. underfitting**

- **Overfitting: the model pays too much attention to training data and fails to generalize to new data.**
- **Underfitting (high bias): the model over-simplifies, often due to an unbalanced dataset or insufficient features.**

# 2 Module 2 — Performance of Distributed Systems

## 2.1 Basics of distributed systems

A distributed system consists of multiple nodes that communicate and coordinate to accomplish tasks; they are designed to be reliable and, crucially, scalable.

- **Scalability:** a measure of how well a system handles increased load. **Throughput** is the number of processes a system handles under load.
- **Scaling up vs. scaling out:** scaling *up* adds hardware (CPU, RAM, disk) to a single server but hits diminishing returns; scaling *out* distributes client requests across multiple nodes.

**Key architectural terms**

- **Stateless services: the server tracks no session data, so different servers can handle different requests from the same user.**
- **Load balancer: a component (often a reverse proxy) that chooses which server handles a request.**
- **Microservices: decomposing a large application into independent parts that communicate via an API, allowing specific functions to scale independently.**

**Performance and efficiency.** Two primary metrics:

- **Latency:** the response time to handle a request and return a result.
- **Bandwidth:** the amount of data transferable per unit of time.

Efficiency comes from distributing load *evenly* — which does not mean giving every server the same number of tasks, since some tasks take longer. Tasks can be assigned at *compile time* (size known in advance) or *run time* (dynamic). Use concurrency and parallelism (threading, multiprocessing) so processors are never idle.

**The CAP theorem**

**Any networked shared-data system can provide only two of the three: Consistency, Availability, Partition tolerance. Because distributed systems inherently include network partitions, developers must typically choose between consistency and availability.**

**Contact tracing app**

**A high-profile app built early in COVID-19 that required massive scale.**

- **The challenge: a "big bang" release with 5 million requests per hour (≈1,400 req/s), sustained up to two hours.**
- **Initial concerns: a monolith using a heavy ORM, synchronous messaging, and no load testing.**
- **Transformations:**
  - ***Automated benchmarking* — a team of 4–6 built a benchmark suite to measure the impact of every change. (Benchmarking = measuring performance under controlled conditions with standardized tests, tracked over time.)**
  - ***Fine-grained control* — moved from automated Spring Data Rest to hand-written controllers.**
  - ***Database optimization* — switched from a heavy ORM (Hibernate) to raw JDBC to avoid "chatty" queries; focused on inserts and selects, avoiding updates/deletes.**

- ***Asynchronous messaging* — moved to RabbitMQ, and from a direct exchange to a consistent hash exchange.**
- ***App Continuum architecture* — moved the monolith toward independently deployable and scalable parts within the same code base.**

## 2.2 Messaging queues — the AMQP 0-9-1 model

AMQP 0-9-1 is a programmable networking protocol where brokers route messages from publishers to consumers. Four primary entities:

- **Exchanges:** act like "post offices," receiving messages and distributing copies to queues based on rules (bindings).
- **Queues:** buffers that store messages until consumers process them.
- **Bindings:** the rules defining the relationship between an exchange and a queue.
- **Consumers:** applications that subscribe to queues to receive and process messages.

### 2.2.1 Exchange types and routing

- **Direct** (default): routes on an exact match between the message's routing key and a queue's binding key.
- **Fanout:** broadcasts a copy to every bound queue, ignoring routing keys.
- **Topic:** matches routing keys against patterns (wildcards) for complex routing.
- **Headers:** routes on multiple message attributes (headers) rather than a routing key.

### 2.2.2 Queue management and best practices

- **Keep queues short:** large backlogs consume RAM and eventually force paging to disk, slowing performance — "a happy Rabbit is an empty Rabbit."
- **Use quorum queues:** recommended over classic mirrored queues for HA and data safety.
- **Lazy queues:** store messages primarily on disk to minimize RAM use, giving more predictable performance during spikes or batch processing.
- **Resource allocation:** queues are single-threaded, so throughput improves by using multiple queues and consumers to leverage multi-core systems.

### 2.2.3 Connections and channels

- **Multiplex with channels:** use channels ("lightweight connections" sharing one TCP connection) instead of many TCP connections.
- **Persistence:** keep connections long-lived — repeatedly opening/closing adds latency from the handshake.
- **Separation of concerns:** use separate connections for publishers and consumers so publisher back-pressure doesn't interfere with consumer acknowledgments.

**Performance vs. high availability**

- **High performance: transient (non-persisted) messages, manual acks disabled, and avoid the overhead of multiple nodes when redundancy isn't required.**
- **High availability: a cluster of at least three nodes across availability zones, durable queues, and persistent messages that survive broker restarts.**
- **Sizing: throughput above 1,000 msg/s needs thorough analysis of CPU, RAM, and disk IOPS. Redundancy is expensive — messages must be replicated across brokers.**

**Consumer control.**

- **Acknowledgments:** consumers use "acks" to tell the broker a message was processed; the broker only removes a message once acked.
- **Prefetch value:** limits how many messages a consumer receives before acking, keeping one consumer from being overwhelmed while others sit idle.

## 2.3 The email verifier codebase

An architecture for high-speed registration and verification of large volumes of users. It mirrors the contact-tracing design and uses a distributed, queue-based approach.

### 2.3.1 Architecture and workflow

1. **Registration request:** a user submits information to the registration app's input endpoint.
2. **Code generation:** the request is placed on a queue; a consumer reads the email and generates a unique confirmation code.
3. **Data storage & messaging:** the code is saved to the registration database and placed on another queue for the notification app.
4. **Notification:** the notification app reads code and email, sends a message via the SendGrid API (or a fake version for testing), and records the notification in its own database.
5. **User confirmation:** the user submits the received code to the confirmation endpoint; if it matches the database record, the email is confirmed and registration is finalized.

**Codebase structure ("app continuum" style, four directories).**

- **Applications:** deployable units actively run (registration and notification servers).
- **Components:** isolated, reusable logic blocks with explicit dependencies.
- **Databases:** tracks all database migrations.
- **Platform support:** supporting services such as a fake SendGrid server used to test high email volumes without sending spam.

**Local setup and execution.**

- **Backing services:** Docker Compose runs RabbitMQ (messaging) and Postgres (databases).
- **Running applications:** managed via Gradle; "fuzzy matching" allows initials (e.g. `a:r:r` for `applications:registration-server:run`).
- **Testing utilities:** a fake SendGrid service prints email messages to the console instead of sending them.

**Registration flow (manual test with curl).** POST to the registration server (port 8081) at `/request-registration` with a JSON email; retrieve the confirmation code from the fake SendGrid console; POST to `/register` with email + code; a `204` status indicates success.

**Why it performs**

- **Asynchronous processing: the queue-based architecture decouples registration requests from generating and sending confirmation codes, so the system accepts high input volume quickly and processes heavier tasks in the background.**
- **High-volume email handling: the fake SendGrid server simulates email sending so capacity can be tested without spam.**
- **Planned benchmarking: benchmarks and performance tests are the critical next step, run across multiple terminals to monitor metrics.**

# 3 Module 3 — Horizontal Distribution of Large Workloads

## 3.1 Performance testing for business requirements

**Non-functional requirements (NFRs)**

**NFRs specify the criteria used to judge how a system should *be* (rather than what it should *do*).**

- **Core attributes — the "ilities": scalability, reliability, availability, security, performance.**
- **Importance: without NFRs it's impossible to design for performance targets, choose testing approaches, or design monitoring.**
- **Performance budgets: translate NFRs into per-component objectives — e.g. a 5-second end-to-end response might allocate 0.2s to the web server and 2s to the database.**
- **Implementation: define NFRs early, by senior people who understand both business needs and the technical landscape.**

### 3.1.1 Performance testing and custom benchmarking

Off-the-shelf tools like k6 and JMeter are excellent for high-volume traffic, but some scenarios need custom benchmarks:

- **Complex flows:** multi-step processes — e.g. an email registration flow where a second request depends on a code from a confirmation email.
- **Integrated mocking:** a custom benchmark can act as a fake server (fake SendGrid) to receive and process notification emails during the test.
- **Configuration:** effective benchmarks use worker pools to hit endpoints rapidly and measure the time to complete a volume of tasks (e.g. 5,000 registrations).

### 3.1.2 The simplified testing pyramid

Traditional pyramids invite unproductive arguments over "unit" vs. "integration." A more effective approach focuses on speed:

- **Fast vs. slow:** categorize tests simply as fast or slow — have as many fast tests as possible and only enough slow tests for full coverage.
- **The cost of slowness:** slow tests dominate a suite's cost by causing unproductive context switching while developers wait.
- **Maintenance:** treat test code like production code — refactor to keep it readable and fast.
- **Limits:** consider hard time limits; if a suite exceeds it (e.g. one minute), fail the build until tests are optimized.

**Test-Driven Development (TDD)**

**Write automated tests *before* production code.**

- **The cycle: red / green / refactor — write a failing test, write the simplest code to pass, then clean up both test and production code.**
- **Advantages: comprehensive coverage, cleaner designs, reduced debugging, increased confidence.**
- **Disadvantages: a larger codebase, a learning curve, and test-suite maintenance overhead.**
- **Test structure: Setup (pre-test state), Execution (trigger behavior), Validation (check results), Cleanup (restore state).**

## 3.2 Message queues continued — the consistent hash exchange

The consistent hash exchange (`x-consistent-hash`) distributes message load across multiple queues via a hashing mechanism.

> **The problem with direct exchanges**
>
> **Adding queues to handle load is manual and intensive. Under heavy traffic, queues grow, RabbitMQ's performance degrades, and a negative feedback loop forms: higher load → larger queues → a slower broker → even harder to drain the queues.**

### 3.2.1 Core mechanism: the hash space

- **Routing keys:** for every message a routing key is computed from the message (body, header, or property) and mapped to a point in the hash space.
- **Binding keys (weights):** queues bind with a binding key that acts as a weight, determining how much of the hash space — and thus traffic — a queue occupies. A queue with binding key "2" occupies twice the space and receives double the traffic of a "1".

**Benefits for scalability.**

- **Publisher independence:** when a new queue is added, the publisher need not be updated or even aware it exists.
- **Automatic reallocation:** binding a new queue makes the exchange reallocate the hash space — existing queues' shares shrink slightly so the new queue immediately takes load.

**Implementation methods.** Three ways to configure what the exchange hashes:

1. **Default (routing key) hashing** — hashes the publisher's routing key.
2. **Header-based hashing** — using the `hash-header` argument, hash a specific header (e.g. `hash-on`) instead of the routing key.
3. **Property-based hashing** — using the `hash-property` argument, hash a message property such as `message_id`.

Code examples exist in Python, Java, Ruby, and Erlang.

### 3.2.2 The hash ring

A conceptual model where a range of hash values is wrapped around into a circle — fundamental to how RabbitMQ's consistent hash exchange and other systems manage horizontal distribution.

- **The circle:** a hash space, often the unit interval $[0, 1)$, wrapped onto a circle.
- **Locating nodes:** each destination (a machine or queue) is assigned a point by its own hash — random positions, not consecutive.
- **Routing messages:** a key is hashed onto the ring; the message is assigned to the first location found moving clockwise.
- **The "end" of the ring:** if a key's hash exceeds all assigned points, it wraps around to the location with the lowest number.

**Dynamic scaling and resilience.**

- **Minimal disruption:** naive modulo-$n$ hashing reallocates nearly all data when a machine is added; on a ring, a new node takes only a small portion from its neighbor — everything else stays put.
- **Automatic reallocation:** existing segments shrink slightly to accommodate a new queue, which takes over some load immediately.

**Ensuring uniform distribution.**

- **Replicas (virtual nodes):** map a machine to multiple points on the ring, improving uniformity and ensuring a new node pulls data from several machines, not just one.
- **Weights in RabbitMQ:** binding keys act as weights — a "2" occupies twice the ring space of a "1" and receives roughly double the traffic.

# 4 Module 4 — Highly Available Distributed Systems

## 4.1 Availability and distributed systems

CAP asserts that a system cannot simultaneously achieve consistency, availability, and partition tolerance when subject to communication failures. In cloud or Kubernetes environments, partition tolerance is non-negotiable, so design becomes a balance between consistency and availability.

- **Consistency-focused** (e.g. Postgres): prioritizes accuracy — essential for routing emergency calls.
- **Availability-focused** (e.g. Cassandra): prioritizes responsiveness — suitable for movie recommendations, where 100% accuracy matters less than a quick response.
- **Hybrid** (e.g. RabbitMQ in front of Postgres): move along the continuum to fit the specific problem.

### 4.1.1 Infrastructure as a dynamical system

Mark Burgess argues IT infrastructure is the largest distributed system of all — a predictable platform for users.

- **Infrastructure consistency:** standards of expectation and conformity.
- **Infrastructure availability:** the platform is ready whenever needed.
- **Infrastructure partition tolerance:** continuing to serve users despite faults, outages, or intended barriers like firewalls and access controls.

Unlike static archives, infrastructure is *dynamical* — monitoring data changes in real time. DNS is a prime example: caching and Time-To-Live (TTL) policies provide availability but yield questionable consistency, because updates take time to propagate.

**The evolution of "atoms" and maintenance.** The units of change have grown from individual configuration files to software packages, golden images, clusters, and entire datacenters. Atom size affects **Mean Time To Repair (MTTR)**: touching a single file gives a short MTTR; re-imaging a whole machine to fix "a light bulb" means long unavailability. Burgess's **Maintenance Theorem** applies Shannon's error-correction to infrastructure: continuous *equilibration*, where an autonomous system (like CFEngine) performs repeated micro-verifications to counteract "noise" or configuration drift. For stability, MTTR must be significantly less than the **Mean Time Before Failure (MTBF)**.

**Policy, autonomy, and "many worlds"**

**Separate *desired state* (policy) from *actual state*.**

- **Slow vs. fast change: policy varies slowly (days/weeks); fluctuations and network failures happen fast (minutes).**
- **3 out of 3 CAP: because policy changes slowly, it can be cached locally at every node — a "many-worlds" environment where each node is autonomously consistent with the policy. In this context a system can achieve all three CAP properties at once, within the engineering tolerances of the repair schedule (e.g. a 5-minute window).**

**Management models: push vs. pull.**

- **Push-based:** uncertain and unreliable (like UDP) — no feedback loop, so you can't define availability or consistency; there's only a point of dispatch.
- **Pull-based / autonomous:** managing "from within" via local agents is superior — it minimizes network latency and partitioning impact, since a local agent maintains policy consistency even after losing contact with a central controller.

Ultimately, infrastructure consistency is not boolean but a matter of *stochastically probable* consistency maintained through continuous autonomous repair.

## 4.2 Databases — giant-scale, highly available systems

### 4.2.1 CAP and theoretical context

CAP identifies a fundamental trade-off between Consistency, Availability, and Partition tolerance: it is impossible to provide all three simultaneously in an unreliable network.

- **Safety vs. liveness:** CAP sits within the broader trade-off between safety ("nothing bad happens" — Consistency) and liveness ("eventually something good happens" — Availability).
- **Impossibility of agreement:** CAP relates to the **FLP impossibility**, which proves fault-tolerant consensus is impossible in an asynchronous system even with one crash failure.
- **Practical responses:** either sacrifice availability (stay consistent but unresponsive during partitions) or sacrifice strong consistency (stay available but return possibly stale data).

### 4.2.2 Metrics for availability: harvest and yield

- **Yield:** the probability of completing a request, often measured in "nines" of uptime.
- **Harvest:** the fraction of total data reflected in a query response — its completeness.
- **Graceful degradation:** many applications keep high yield during faults by sacrificing harvest — e.g. a search engine returning results from 99% of its database if one node fails.

**The DQ principle**

**Data per query ($D$) $\times$ Queries per second ($Q$) approaches a physical bottleneck (usually bandwidth or I/O):**

$$D \times Q \approx \text{constant}.$$

**This lets engineers predict how faults or optimizations affect performance. For evolving systems, improving MTTR is often more effective than improving MTBF — it's easier to achieve and has the same impact on uptime. Giant-scale services use clusters of commodity servers, load managers (Layer-4 and Layer-7 switches), and persistent data stores.**

**Amazon's Dynamo**

**Amazon's highly available key-value store, designed for massive scale while staying "always-on" for critical services like the shopping cart.**

- **Eventual consistency: prioritizes availability for writes, using *vector clocks* to capture causality and allow asynchronous update propagation.**
- **Conflict resolution: conflicts are pushed to the read operation so writes are never rejected; resolution is handled by the data store ("last write wins") or application logic (merging shopping carts).**
- **Techniques: consistent hashing with virtual nodes (uniform load over heterogeneous hardware); quorum systems tuning $N$ (replicas), $R$ (min read nodes), $W$ (min write**

> **nodes); gossip-based protocols for decentralized failure detection and membership; hinted handoff and anti-entropy (via Merkle trees) for transient and permanent failures.**

**Engineering strategies for evolution and faults.**

- **Application decomposition:** breaking a system into independent subsystems (billing vs. session management) lets each part make different CAP trade-offs.
- **Orthogonal mechanisms:** timeouts, retries, and sandboxing — independent of application logic — simplify providing scalability and robustness.
- **Online evolution:** upgrade live systems via rolling upgrades (one node at a time), fast reboots (all at once), or the "big flip" (half the cluster at once) to minimize impact on yield or harvest.

### 4.2.3 Foundational papers

1. **Dynamo: Amazon's Highly Available Key-value Store** (DeCandia et al., 2007) — decentralized, eventually consistent storage; consistent hashing, vector clocks, quorum-based protocols.
2. **Perspectives on the CAP Theorem** (Gilbert & Lynch, 2012) — CAP as a fundamental trade-off between safety (consistency) and liveness (availability).
3. **Harvest, Yield, and Scalable Tolerant Systems** (Fox & Brewer, 1999) — introduces yield and harvest and how systems gracefully degrade during faults.
4. **Lessons from Giant-Scale Services** (Brewer, 2001) — the DQ principle and strategies for online evolution like rolling upgrades and the "big flip".

Course 3 of 3 — Capstone

# Applications of Software Architecture for Big Data

Software Architecture for Big Data Specialization
University of Colorado Boulder

Compiled by Ioana-Raluca Tiriac

# Contents

# 1 Module 1 — Intro & Project Overview

## 1.1 The course project

The project is graded in tiers. The minimum ("C level") is the seven core requirements; "A level" encompasses everything for a complete project.

| C level (7 — minimum) | B level (adds) | A level (adds) |
|---|---|---|
| Web application (basic form, reporting) | Integration tests | Event collaboration (messaging) |
| Data collection | Mock objects, fakes, or spies | Continuous delivery |
| Data analyzer | Continuous integration | |
| Unit tests | Production monitoring / instrumenting | |
| Data persistence (any data store) | | |
| REST collaboration (internal or API endpoint) | | |
| Product environment | | |

B level = the seven C-level items *plus* the B column; A level = all of B *plus* the A column (13 items total).

### 1.1.1 Project selection and examples

Choose something you enjoy — a hobby, a work task, or a concept from another class. Three examples that fit the required architecture:

- **Land Cruiser Finder:** scraped data from Toyota dealerships, analyzed it for specific models and colors, and displayed results on a webpage with alerts.
- **Provenance:** collected data from InfoQ to analyze whether articles were original or republished, focusing on trusted authors rather than publishers.
- **Instagram RSS Feed:** scraped Instagram's private API (using a Raspberry Pi to bypass cloud-provider blocks), stored unstructured data in Redis, and used a background worker to process it for an RSS reader.

> **Whiteboard architecture — three free-running processes**
>
> 1. **Data collector: reaches out to public APIs (Twitter, weather) or scrapes to gather data. Often scheduled with cron: wake up, collect, save to a database, shut down.**
> 2. **Data analyzer: retrieves raw data from the database to perform analysis — sentiment analysis, image processing, or rolling data up into a more useful reporting format.**
> 3. **Web application: the interface where the end user interacts, displaying the data the analyzer has processed.**

**System integration and communication.**

- **Database:** the central repository where the collector drops off data and the analyzer retrieves it.

- **Message queue:** lets components signal one another — e.g. a user action in the web app can trigger the collector or analyzer. Preferred over polling intervals because it moves data through the pipeline faster.

# 2 Module 2 — MVP & Dev Environment

## 2.1 Getting started with an MVP

> **Minimum Viable Product (MVP)**
>
> **The smallest possible version of a product that can be launched quickly to gather user feedback.**
>
> - **Iterative value: instead of building components with no standalone value (a car wheel without a body), solve a core problem (transportation) in stages — start with something as simple as a skateboard.**
> - **Feedback loops: deploy to production quickly to get feedback, adjust, then build the next iteration (scooter, bike, car) from real user needs.**
> - **Inverted process: deploy first, then write automated tests, then develop — ensuring a daily feedback cycle.**

### 2.1.1 Source-code branching patterns

- **Mainline (trunk):** a single shared branch where all current development is integrated; developers pull and push to keep in sync.
- **Feature branching:** work on a dedicated branch for a feature and merge back once complete.
- **Release and maturity branches:** a *release branch* stabilizes a version for release; a *maturity branch* represents the latest "production-ready" code.

### 2.1.2 Continuous integration and deployment

- **Core practices:** a single source repository, an automated self-testing build, and every developer committing to the mainline daily.
- **Benefits:** avoids "integration hell," detects bugs faster, and sustains productivity through constant refactoring.
- **Continuous delivery:** the latest changes are always in a state where they could reach users in production at any time.

**Practical project setup.**

- **Web framework:** Flask, for simple apps that handle input and display data.
- **Code hosting:** GitHub for version control and collaboration.
- **Cloud deployment:** Heroku as a Platform-as-a-Service for a public URL.
- **CI tools:** GitHub Actions or Travis CI to automate testing and deployment.

## 2.2 The development environment

### 2.2.1 Testing methodologies

- **Unit testing:** test individual components (classes, methods) in isolation. In Python, the `unittest` framework has test cases subclass `unittest.TestCase` and use assertions like `assertEqual`.
  - *Mock objects:* simulate a controlled response for fluctuating external data (e.g. currency rates) using `MagicMock`.

  - *Test fixtures:* `setUp()` prepares the environment before a test; `tearDown()` cleans up afterward.
- **Integration testing:** tests interoperability between modules (client and server over a network). For the project, integration tests should involve core components — the data collector, data analyzer, or a messaging queue.

#### 2.2.2 Continuous integration

Run the full test suite automatically on every commit to a shared repository like GitHub.

- **Automation with GitHub Actions:** a `build.yml` file defines workflows that trigger on every "push".
- **Goal:** ensure all tests pass before integration, catching bugs early. Tools like Gradle can run these tests before building the final artifact.

#### 2.2.3 Continuous delivery

Automatically push a tested application to a review or production environment — a two-step pipeline:

1. **Packaging:** package the app into a Docker container and publish to a registry (e.g. Google's container registry) using tools like Cloud Build.
2. **Deployment:** deploy the container to a target platform — Google Cloud Run, Amazon, Azure, or Heroku — using automated commands and secrets stored in GitHub.

All three processes (web app, data analyzer, data collector) should be wired into the CI/CD pipeline so they are automatically tested and redeployed on every change.

## 3 Module 3 — Affixing Features

### 3.1 Data persistence strategies

- **Data Gateway pattern:** decouples the application from the database, managing storage and retrieval of records.
- **Records as data classes:** simple data classes (e.g. a `Product` with an id, name, and quantity) representing the information handled.
- **CRUD operations:** a data gateway typically implements Create, Read, Update, Delete for configuration or user data.
- **Template pattern:** commonly used for database interactions, mapping database rows directly to application objects (the "mapping" component of an ORM).

> **The Milk Problem**
>
> **Illustrates a `ProductDataGateway` that increments or decrements the quantity of milk gallons stored in a database.**

### 3.2 External data collection (REST APIs)

- **Fetching data:** applications use a "worker" class to reach external REST endpoints.
- **Tools and formats:** in Java, a `RestTemplate` (similar to curl) executes GET or POST commands; in Python, the `requests` library fetches JSON (e.g. current temperature from a weather API).
- **Data marshalling:** when receiving XML or RSS, the app must "marshall" the content into an object structure before saving specific fields (like article titles).

- **Analysis:** beyond collection, APIs can serve analyzed data — e.g. a Flask endpoint that computes the average temperature over a stored date range.

### 3.3 Automation and scheduling

Automate collection with cron, cloud-based schedulers (e.g. Heroku Scheduler), or custom workflow-support classes acting as a task runner.

> **Testing best practices**
>
> - **Database testing: perform "developer tests" on the database layer *without* mocking, so the actual SQL (like insert statements) is tested before production.**
> - **Worker / API testing: when testing collection from external sources, mock the API response — this keeps tests fast and predictable rather than dependent on live URLs.**

## 4 Module 4 — Scaling Your MVP & Wrapping Up

### 4.1 Production readiness

#### 4.1.1 Health checks and metrics

- **Health checks:** a simple endpoint (e.g. `/healthcheck`) returning whether the app is "healthy" — critical for platforms like Kubernetes to decide if an app can receive traffic or needs restarting.
- **Basic metrics:** register "meters" or counters with a metrics registry to track activity, such as requests to specific controllers.
- **Infrastructure:** Prometheus collects metrics (a specialized database); Grafana visualizes them.

#### 4.1.2 Desirable features of a monitoring strategy

- **Speed and freshness:** data should be available quickly — anything more than four to five minutes stale can significantly hinder incident response.
- **Calculations and granularity:** support monotonically incrementing counters and statistical functions like percentiles (50th, 95th, 99th) rather than just means, which can mask bad behavior.
- **Purposeful metrics:** each metric should serve a goal — alerting metrics should change dramatically only when there's a problem, while debugging metrics should reveal the cause.

#### 4.1.3 Metrics vs. logs

- **Metrics:** numerical measurements at regular intervals — near real-time but less granular; ideal for alerts and dashboards.
- **Logs:** append-only records of events — highly granular and better for root cause, but delayed and more expensive to process.
- **Best practice:** use metrics-based alerting even for single exceptional events by incrementing a counter, keeping alert configuration centralized.

#### 4.1.4 Advanced alerting (SLO-based)

The goal is to notify engineers of "significant events" that threaten the error budget. Multi-window, multi-burn-rate alerts are the most viable.

- **Burn rate:** how fast a service consumes its error budget relative to the SLO. A burn rate of 1 exhausts the budget exactly at the end of the SLO period.

- **The multi-window approach:** check both a *long* window (to confirm the event is significant) and a *short* window (to verify it's still active).
- **Recommended parameters:** for a 99.9% SLO, a page-level alert might trigger when 2% of the budget is consumed in one hour (a 14.4× burn rate).

#### 4.1.5 Management and scalability

- **Configuration as code:** store monitoring and alerting config in version control for history and code review.
- **Consistency and coupling:** a centralized framework helps engineers ramp up across teams; keep components (collection, storage, alerting, visualization) loosely coupled so they can be swapped.
- **Handling low traffic:** simple error rates can cause false pages for low-request services — generate synthetic traffic, combine related services, or lower the SLO to reflect real user impact.
- **Alerting at scale:** group requests into buckets by availability requirement (e.g. `CRITICAL`, `HIGH_FAST`, `LOW`) instead of unique parameters per service.

### 4.2 Asynchronous events — event collaboration

> **Event collaboration**
>
> **An architectural pattern where components collaborate by sending events when their internal state changes — fundamentally different from the request–response style, where one component requests information or actions from another.**

#### 4.2.1 Core principles and mechanics

- **Broadcasting vs. direct requests:** components "speak when they have something to say" — senders broadcast events without knowing who is interested.
- **Handling queries:** rather than querying a central source, components listen to broadcast data events and maintain their own local copy of what they need.
- **Handling commands:** commands are phrased as events that have occurred, inviting interested parties to respond. This promotes decentralization but carries an *implied responsibility* — the system must ensure some component actually acts on the event.
- **State management:** responsibility shifts from a single "home" to the users of the data, giving *controlled data replication* — each component stores only the subset it requires.

**Advantages.**

- **Loose coupling:** add new components without modifying existing ones, as long as they listen to the established event bus.
- **Simplicity and focus:** components stay simple — they only know the events they consume and emit.
- **Resilience:** components keep operating even if external communication is lost, since they hold the data they need (though possibly stale).

> **Challenges and risks**
>
> - **Event cascades: one event triggers further events, producing side effects that are hard to predict because the full logic chain isn't visible in a single context.**
> - **Implicit complexity: interactions aren't explicit in the source, so the flow of logic is hard**

**to find, understand, and debug.**

- **Data replication: multiple copies increase storage needs and the risk of components working with out-of-date information if they miss updates.**

**Technical implementation.** Practical implementations use message-queuing software as an asynchronous data structure:

- **Message broker:** a middleman between producers (which populate the queue with events) and consumers (which subscribe and retrieve data).
- **Tools:** platforms like RabbitMQ handle these transactions, often via language-specific libraries such as `pika` for Python.